

Experiment: Bank Account Management System

■ Basic Information

Experiment No: 01

Date: 11/09/2026

Title: Bank Account Management System using Classes and Objects

■ Objective / Aim

To design and implement a **Bank Account Management System** in Java using **classes and objects**, demonstrating object-oriented programming concepts such as attributes, methods, object creation, deposits, withdrawals, interest calculation, and fund transfers.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

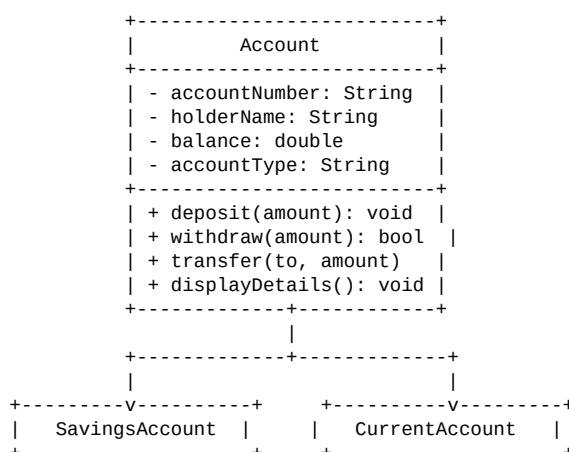
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a base class named **Account**.
2. Define attributes such as account number, account holder name, balance, and account type.
3. Create a **SavingsAccount** class with an additional interestRate attribute.
4. Create a **CurrentAccount** class with an additional overdraftLimit attribute.
5. Define methods in the Account class for deposit(), withdraw(), transfer(), and displayAccountDetails().
6. Define calculateInterest() in SavingsAccount to calculate interest based on the current balance.
7. Override withdraw() in CurrentAccount to allow withdrawals up to the permitted overdraft limit.
8. Create objects for savings and current accounts in main().
9. Perform deposits, withdrawals, transfers, and interest calculations.
10. Display the updated account details and transaction results.

■ UML Class Diagram



- interestRate		- overdraftLimit	
+-----+		+-----+	
+ calculateInterest		+ withdraw()	
+-----+		+-----+	

Relationship: SavingsAccount and CurrentAccount inherit from the Account class.

■ Source Code

```
class Account {
    protected String accountNumber;
    protected String holderName;
    protected double balance;
    protected String accountType;

    Account(String accountNumber, String holderName,
            double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: Rs." + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: Rs." + amount);
            return true;
        }

        System.out.println("Insufficient balance.");
        return false;
    }

    public void transfer(Account target, double amount) {
        if (withdraw(amount)) {
            target.deposit(amount);
            System.out.println("Transferred: Rs." + amount
                    + " to " + target.accountNumber);
        } else {
            System.out.println("Transfer failed.");
        }
    }

    public void displayAccountDetails() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Holder Name : " + holderName);
        System.out.println("Account Type : " + accountType);
        System.out.printf("Balance : Rs.%.2f%n", balance);
    }
}

class SavingsAccount extends Account {
    private double interestRate;

    SavingsAccount(String accountNumber, String holderName,
            double balance, double interestRate) {
        super(accountNumber, holderName, balance, "Savings");
        this.interestRate = interestRate;
    }

    public double calculateInterest() {
        return balance * interestRate / 100;
    }

    @Override
    public void displayAccountDetails() {
        super.displayAccountDetails();
        System.out.println("Interest Rate : " + interestRate + "%");
        System.out.printf("Interest : Rs.%.2f%n",
                calculateInterest());
    }
}
```

```

}

class CurrentAccount extends Account {
    private double overdraftLimit;

    CurrentAccount(String accountNumber, String holderName,
                    double balance, double overdraftLimit) {
        super(accountNumber, holderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance + overdraftLimit) {
            balance -= amount;
            System.out.println("Withdrawn: Rs." + amount);
            return true;
        }

        System.out.println("Withdrawal exceeds overdraft limit.");
        return false;
    }

    @Override
    public void displayAccountDetails() {
        super.displayAccountDetails();
        System.out.printf("Overdraft Limit: Rs.%.2f%n", overdraftLimit);
    }
}

public class BankAccountDemo {
    public static void main(String[] args) {
        SavingsAccount savings =
            new SavingsAccount("SA101", "Rahul", 10000, 5.0);

        CurrentAccount current =
            new CurrentAccount("CA201", "Priya", 15000, 5000);

        System.out.println("===== SAVINGS ACCOUNT =====");
        savings.displayAccountDetails();

        System.out.println("\nDepositing Rs.2000...");
        savings.deposit(2000);

        System.out.println("\nWithdrawing Rs.1500...");
        savings.withdraw(1500);

        System.out.println("\n===== AFTER TRANSACTIONS =====");
        savings.displayAccountDetails();

        System.out.println("\n===== CURRENT ACCOUNT =====");
        current.displayAccountDetails();

        System.out.println("\nWithdrawing Rs.18000...");
        current.withdraw(18000);

        System.out.println("\n===== AFTER WITHDRAWAL =====");
        current.displayAccountDetails();

        System.out.println("\n===== TRANSFER =====");
        System.out.println("Transferring Rs.2000 from Savings to Current...");
        savings.transfer(current, 2000);

        System.out.println("\n===== FINAL ACCOUNT DETAILS =====");

        System.out.println("\nSavings Account:");
        savings.displayAccountDetails();

        System.out.println("\nCurrent Account:");
        current.displayAccountDetails();
    }
}

```

■ Compilation & Execution Commands

Save the source code as: BankAccountDemo.java

```

javac BankAccountDemo.java
java BankAccountDemo

```

■■ Output Screen

```

===== SAVINGS ACCOUNT =====
Account Number : SA101
Holder Name    : Rahul
Account Type   : Savings
Balance        : Rs.10000.00
Interest Rate  : 5.0%
Interest       : Rs.500.00

Depositing Rs.2000...
Deposited: Rs.2000.0

Withdrawing Rs.1500...
Withdrawn: Rs.1500.0

===== AFTER TRANSACTIONS =====
Account Number : SA101
Holder Name     : Rahul
Account Type    : Savings
Balance         : Rs.10500.00
Interest Rate   : 5.0%
Interest        : Rs.525.00

===== CURRENT ACCOUNT =====
Account Number : CA201
Holder Name     : Priya
Account Type    : Current
Balance         : Rs.15000.00
Overdraft Limit: Rs.5000.00

Withdrawing Rs.18000...
Withdrawn: Rs.18000.0

===== AFTER WITHDRAWAL =====
Account Number : CA201
Holder Name     : Priya
Account Type    : Current
Balance         : Rs.-3000.00
Overdraft Limit: Rs.5000.00

===== TRANSFER =====
Transferring Rs.2000 from Savings to Current...
Withdrawn: Rs.2000.0
Deposited: Rs.2000.0
Transferred: Rs.2000.0 to CA201

===== FINAL ACCOUNT DETAILS =====

Savings Account:
Account Number : SA101
Holder Name     : Rahul
Account Type    : Savings
Balance         : Rs.8500.00
Interest Rate   : 5.0%
Interest        : Rs.425.00

Current Account:
Account Number : CA201
Holder Name     : Priya
Account Type    : Current
Balance         : Rs.-1000.00
Overdraft Limit: Rs.5000.00

```

■ Result

The **Bank Account Management System** was successfully implemented in Java using **classes and objects**. The program demonstrates the creation of `SavingsAccount` and `CurrentAccount` objects and performs deposits, withdrawals, interest calculation, and fund transfers successfully.